

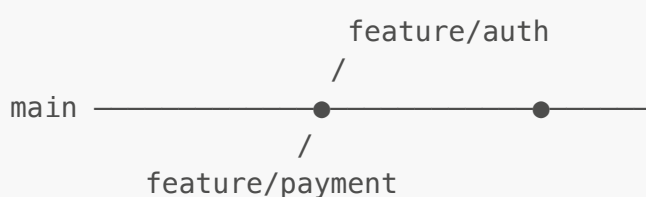
Absolutely. For **Day 7**, I'd focus on being able to **explain the concept, show the workflow, and answer scenario-based interview questions**.

Day 7 — Git, Testing & Mock Interview

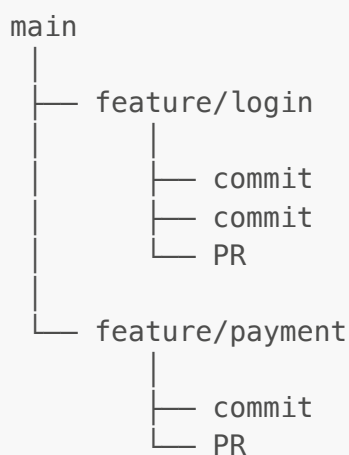
1. Git Branching

Core idea

Git branching allows multiple developers to work on different features without modifying the main branch directly.



Typical workflow:



Interview answer

I normally create a feature branch from the latest main/develop branch, make isolated commits, push the branch and create a pull request. This keeps production code stable and allows code review before merging.

Commands

```
git checkout main
git pull origin main

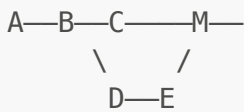
git checkout -b feature/login
```

```
git add .  
git commit -m "Add login API"  
  
git push -u origin feature/login
```

2. Merge vs Rebase

This is one of the most commonly asked Git questions.

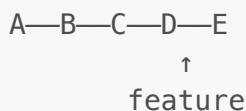
Merge



Merge combines histories and creates a merge commit.

```
git checkout main  
git merge feature/login
```

Rebase



Rebase moves your commits on top of the latest base.

```
git checkout feature/login  
git rebase main
```

Main difference

Merge	Rebase
Preserves history	Rewrites commit history
Creates merge commit	Usually linear history
Safer for shared branches	Good for private feature branches

Merge

Doesn't rewrite existing commits

Rebase

Changes commit hashes

Golden interview answer

I prefer rebase for my local/private feature branch when I want a clean linear history. I avoid rebasing commits that other developers are already working on because rebase rewrites history. For shared branches, merge is generally safer.

Critical question

Can you rebase a public branch?

Technically yes, but I generally avoid it because it rewrites commit history and can cause problems for other developers who already based their work on those commits.

3. Conflict Resolution

Imagine:

```
main:
A — B — C

feature:
A — B — D
```

Both modified the same code.

Git:

```
<<<<<<< HEAD
main version
=====
feature version
>>>>>>> feature/login
```

Resolution process

```
git pull
```

If conflict occurs:

```
git status
```

Open the conflicting files and decide which implementation should remain.

Then:

```
git add .  
git commit
```

For rebase:

```
git add .  
git rebase --continue
```

To cancel:

```
git rebase --abort
```

Interview answer

I don't blindly choose ours or theirs. First I understand why both changes were made, determine the intended behavior, resolve the code accordingly, run tests, and then complete the merge or rebase.

That sounds much stronger than:

"I remove the conflict markers."

4. Revert vs Reset

Very important.

Reset

Moves the branch pointer.

Before:

```
A — B — C — D  
                ↑  
            HEAD
```

```
git reset --hard B
```

A — B

Commands:

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

Revert

Creates a new commit that reverses an earlier commit.

```
A — B — C — D — E
          ↑
        revert C
```

Interview answer

I use `git revert` when the commit has already been pushed/shared because it preserves history.
I use `git reset` mainly for local commits that haven't been shared.

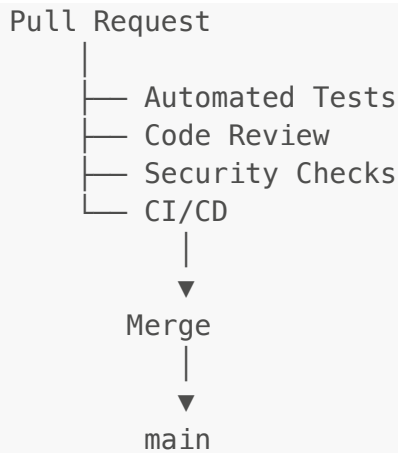
Remember

```
Shared history → revert
Private history → reset
```

5. Pull Requests

A good PR workflow:

```
Developer
  |
  ▼
Feature Branch
  |
  ▼
Push
  |
  ▼
```



What should a good PR contain?

- Clear title
- Description
- Why the change was required
- What changed
- Testing information
- Screenshots if frontend
- API examples if relevant
- Known limitations
- Small focused commits

Interview question

"What do you look for while reviewing a PR?"

Answer:

I check correctness first, then architecture, readability, error handling, security, performance, test coverage and whether the change follows existing project conventions.

For your FastAPI work, specifically mention:

I also check whether business logic has accidentally been placed inside the router, whether database queries can cause N+1 problems, whether Pydantic schemas are appropriate, and whether external API calls have proper timeout and error handling.

6. Unit Testing

A unit test tests a small isolated piece of logic.

For example:

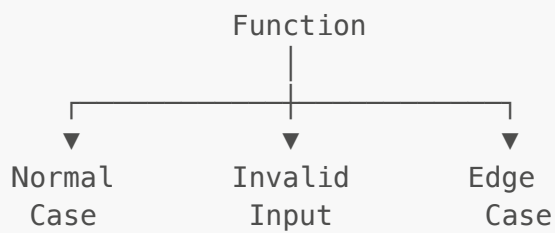
```
def calculate_discount(price, discount):
```

```
return price - (price * discount / 100)
```

Test:

```
def test_calculate_discount():  
    assert calculate_discount(1000, 10) == 900
```

What should you test?



Example:

```
price = 1000  
discount = 10  
→ 900  
  
price = 1000  
discount = 0  
→ 1000  
  
price = 1000  
discount = 100  
→ 0  
  
price = 1000  
discount = -10  
→ validation/error
```

For your FastAPI projects

You should be comfortable with:

```
pytest  
+  
FastAPI TestClient  
+  
mocking external services
```

Example:

```
def test_create_company(client):  
    response = client.post(  
        "/companies",  
        json={"name": "Infosys"}  
    )  
  
    assert response.status_code == 201
```

7. API Testing

For API testing, don't test only 200.

Think:

```
Request  
├── Valid request  
├── Missing field  
├── Invalid field  
├── Unauthorized  
├── Forbidden  
├── Not found  
├── Duplicate  
├── Large payload  
└── External service failure
```

Example

For:

```
POST /companies
```

Test:

```
201 → Created  
  
400 → Invalid request  
  
401 → Unauthenticated  
  
403 → Unauthorized
```



```
409 → Duplicate
422 → Validation failure
500 → Unexpected server error
```

8. Mocking

This is especially important for **your YouTube/Grok/RAG work**.

Suppose your service calls:

```
YouTube API
  ↓
YouTube Connector
  ↓
Your Service
```

Don't call the real YouTube API in every unit test.

Instead:

```
Test
|
▼
Mock YouTube Client
|
▼
YouTube Connector
|
▼
Your Service
```

Why?

Unit tests should be deterministic, fast and independent of external services.

Example with `unittest.mock`:

```
@patch("app.clients.youtube_client.YouTubeClient.search_videos")
def test_search(mock_search):
    mock_search.return_value = [
        {"id": "abc123"}
    ]
```

```
result = service.search("Infosys")

assert result[0]["id"] == "abc123"
```

9. Debugging

Use a structured process.

```
graph TD
    Bug --> Reproduce
    Reproduce --> ReadLogs[Read logs/error]
    ReadLogs --> IdentifyLayer[Identify failing layer]
    IdentifyLayer --> InspectInputs[Inspect inputs]
    InspectInputs --> InspectDB[Inspect database/API]
    InspectDB --> FindRootCause[Find root cause]
    FindRootCause --> Fix
    Fix --> Test
    Test --> RegressionTest[Regression test]
```

Don't say:

"I put print statements."

Say:

I first reproduce the issue, identify the failing layer, inspect logs and request/response data, isolate the root cause, implement the smallest appropriate fix, and then add a regression test.

10. Logging

For production systems, don't rely on:

```
print("something happened")
```

Use structured logging.

```
logger.info(  
    "Company analysis completed",  
    extra={  
        "company": company_name,  
        "duration_ms": duration,  
    }  
)
```

Useful levels:

```
DEBUG  
↓  
INFO  
↓  
WARNING  
↓  
ERROR  
↓  
CRITICAL
```

What should you log?

Good:

```
request_id  
user_id  
service  
endpoint  
duration  
status_code  
error type  
external provider
```

Avoid logging:

```
password
JWT/token
API keys
credit card information
sensitive personal information
```

Your architecture

For your microservices:

```
FastAPI Service
  |
  ▼
Structured Logs
  |
  ▼
Centralized Logging
  |
  ├── Search
  ├── Filter
  └── Monitor
```

11. Edge Cases

This is one of the biggest DSA + backend interview topics.

For every feature, ask:

Input

```
null?
empty?
negative?
zero?
huge?
wrong type?
duplicate?
```

Database

```
record doesn't exist?
duplicate record?
```

```
transaction failure?  
connection failure?
```

API

```
timeout?  
rate limit?  
third-party failure?  
invalid response?
```

Example: YouTube API

Normal:

```
Company → Videos → Statistics
```

Edge cases:

```
Company doesn't exist  
↓  
No videos configured  
↓  
Video deleted  
↓  
Video is private  
↓  
YouTube API timeout  
↓  
API quota exceeded  
↓  
Malformed response
```

This connects directly to your actual project experience.

Mock Interview — Day 7

Now these are the questions I'd expect for your experience.

Git

Q1. What is the difference between merge and rebase?

Q2. When would you use `git revert` instead of `git reset`?

Q3. You have a conflict during rebase. Walk me through exactly what you do.

Q4. A developer accidentally pushed a secret/API key to Git. What do you do?

Q5. Why should you avoid rebasing a shared branch?

Testing

Q6. What is the difference between unit, integration and end-to-end testing?

Q7. How would you test a FastAPI endpoint?

Q8. How would you mock a YouTube API call?

Q9. What should you test when an API returns a **200** response?

Q10. How do you test database-related code?

Debugging

Q11. Your FastAPI endpoint suddenly became 5x slower. How would you investigate?

Strong answer structure:

```
API latency
  ↓
Application logs
  ↓
DB query time
  ↓
External API latency
  ↓
CPU / Memory
  ↓
Network
  ↓
Recent deployment
```

Logging

Q12. What information should never be logged?

Q13. How would you debug an error occurring only in production?

Q14. Why is structured logging better than print statements?

Edge Cases

Q15. Your API receives the same payment callback twice. What happens?

This is especially relevant to your **payment/callback work**.

A strong answer:

I would make the callback idempotent. Before processing the payment, I'd check whether the transaction has already been processed using a unique transaction/reference ID. If it has, I return the existing result rather than processing it again.

The 5 answers you absolutely must master

For your interview, don't just memorize definitions. Be able to explain these deeply:

1. Merge vs Rebase

```
Merge    → preserve history
Rebase   → rewrite history
```

2. Revert vs Reset

```
Shared commit → revert
Local commit  → reset
```

3. Unit vs Integration

```
Unit
Function → isolated

Integration
API → Service → DB

E2E
User → Frontend → API → DB
```

4. Mocking

```
Real external API ❌
    ↓
Mock ✅
    ↓
Deterministic test
```

5. Debugging

```
Reproduce
↓
Observe
↓
Isolate
↓
Find root cause
↓
Fix
↓
Test
↓
Prevent regression
```

For your profile, Day 7 should be less about memorizing Git commands and more about demonstrating engineering judgment: *why you chose an approach, how you safely handled production problems, and how you proved the fix worked.*